

Name: Ruiyi Sun Assignment number: 1

Github repo link: <https://github.com/lazytitan234/CWM-FDI/tree/main/assignment1>

Q1: Time measurement using “time”

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

real    0m0.233s
user    0m0.226s
sys     0m0.007s
```

Q1: Time measurement using “/usr/bin/time -v

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ /usr/bin/time -v time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
0.23user 0.01system 0:00.25elapsed 98%CPU (0avgtext+0avgdata 13272maxresident)k
164inputs+0outputs (0major+1945minor)pagefaults 0swaps
Command being timed: "time python3 matmul_slow.py"
User time (seconds): 0.23
System time (seconds): 0.02
Percent of CPU this job got: 98%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.26
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 13272
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 2035
Voluntary context switches: 47
Involuntary context switches: 4
Swaps: 0
File system inputs: 232
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Analysis for Q1:

Real time is the longest at 0.233s, followed by user time at 0.226s, with system time being the fastest at 0.007s.

Q2: Running 5 times and recording wall clock timings

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ for i in 1 2 3 4 5; do /usr/bin/time -f "%e" python3 matmul_slow.py; done
n=128 reps=2 checksum=107389.290000
0.23
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
```

Analysis for Q2:

I would report the median time as the summary statistics instead of mean or minimum time. There's inherently some fluctuations in terms of time taken to execute the code, so there's not much point in reporting the minimum as this is rarely achieved and not a very useful statistic. Mean is slightly better than the minimum but it has the problem that extreme timing readings (anomalously small or large timings) could skew it. Median time spent gives a much better picture on the central tendency of the timing.

Q3: Results of running sudo perf

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py':

      813855462      cycles
      2298820009      instructions          #    2.82  insn per cycle
       745565      cache-misses

0.228498576 seconds time elapsed

0.216682000 seconds user
0.010983000 seconds sys
```

Q4: Results of running sudo perf 5 times

```

ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

Performance counter stats for 'python3 matmul_slow.py' (5 runs):

      804819264      cycles
    ( +- 0.41% )
    2295023515      instructions          #    2.85  insn per cycle
    ( +- 0.04% )
      562137      cache-misses
    ( +- 3.12% )

    0.22950 +- 0.00466 seconds time elapsed ( +- 2.03% )

```

Q3 and 4 analysis:

Based on the results of 5 runs, we averaged 2.85 instructions per cycle. This means the CPU is capable of executing more than 1 instruction per clock cycle. The fact that it is not a whole number also suggests that the number of instructions executed in each clock cycle is not the same (some cycles might execute 2 whilst others might execute 3 for example).

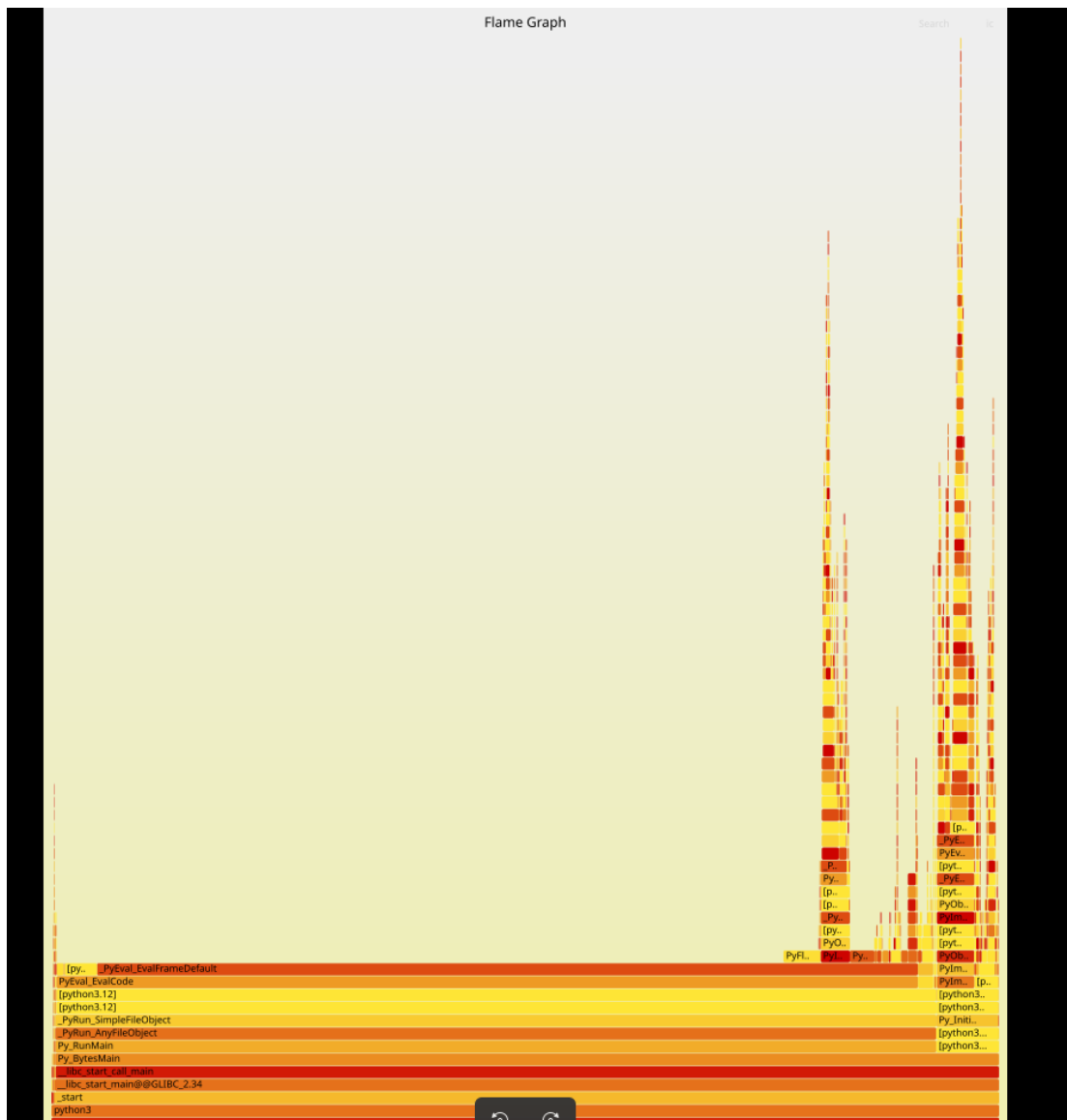
There were about 562k cache misses on average across 5 cycles. The total time it takes to run code is determined by multiple factors, one of which is the time taken to retrieve data from memory. This is why we do cacheing to reduce the time taken to retrieve data. We want as few cache misses as possible, as this implies we are retrieving data from our cache instead of main memory, which is much faster.

Q5: Hotspot discovery

Samples: 919 of event 'cycles:P', Event count (approx.): 807133559					
	Children	Self	Command	Shared Object	Symbol
+	99.81%	0.00%	python3	python3.12	[.] _start
+	99.81%	0.00%	python3	libc.so.6	[.] __libc_start_main@@GLIBC_2.34
+	99.75%	0.00%	python3	libc.so.6	[.] __libc_start_call_main
+	99.75%	0.00%	python3	python3.12	[.] Py_BytesMain
+	95.91%	0.00%	python3	python3.12	[.] PyEval_EvalCode
+	92.89%	76.27%	python3	python3.12	[.] _PyEval_EvalFrameDefault
+	92.72%	0.00%	python3	python3.12	[.] Py_RunMain
+	92.72%	0.00%	python3	python3.12	[.] _PyRun_AnyFileObject
+	92.72%	0.00%	python3	python3.12	[.] _PyRun_SimpleFileObject
+	90.86%	0.00%	python3	python3.12	[.] 0x000000000006b5263
+	90.75%	0.00%	python3	python3.12	[.] 0x00000000000608b52
+	8.81%	0.00%	python3	python3.12	[.] PyImport_ImportModuleLevelObject
+	8.81%	0.00%	python3	python3.12	[.] PyObject_CallMethodObjArgs
+	8.81%	0.00%	python3	python3.12	[.] 0x0000000000054a087
+	8.48%	0.00%	python3	python3.12	[.] 0x0000000000058206d
+	7.76%	0.00%	python3	python3.12	[.] 0x000000000005d36ac
+	7.03%	0.00%	python3	python3.12	[.] 0x000000000006bc965
+	6.97%	0.00%	python3	python3.12	[.] 0x000000000006bca35
+	6.97%	0.00%	python3	python3.12	[.] Py_InitializeFromConfig
+	5.79%	0.00%	python3	python3.12	[.] 0x000000000005d39f4
+	5.78%	0.00%	python3	python3.12	[.] 0x000000000006b0bc4
+	5.47%	0.00%	python3	python3.12	[.] PyObject_Vectorcall
+	5.15%	0.00%	python3	python3.12	[.] PyImport_Import
+	5.15%	0.00%	python3	python3.12	[.] PyObject_CallFunction
+	5.05%	0.00%	python3	python3.12	[.] PyImport_ImportModule
+	4.14%	0.00%	python3	python3.12	[.] 0x000000000006b1132
+	3.09%	1.87%	python3	python3.12	[.] PyFloat_FromDouble
+	2.77%	0.00%	python3	python3.12	[.] _PyObject_MakeTpCall
+	2.59%	0.00%	python3	python3.12	[.] 0x000000000005d3759
+	2.59%	0.00%	python3	python3.12	[.] PyRun_StringFlags
+	2.36%	0.00%	python3	python3.12	[.] 0x00000000000599c17
+	2.31%	1.87%	python3	python3.12	[.] PyLong_FromLong
+	2.25%	0.00%	python3	python3.12	[.] 0x000000000005d2acb
+	1.90%	0.00%	python3	[kernel.kallsyms]	[k] entry_SYSCALL_64_after_hwframe
+	1.90%	0.02%	python3	[kernel.kallsyms]	[k] do_syscall_64
+	1.89%	0.00%	python3	[kernel.kallsyms]	[k] x64_sys_call
+	1.65%	0.00%	python3	python3.12	[.] 0x000000000006b542c
+	1.65%	0.00%	python3	python3.12	[.] 0x000000000006b55bd
+	1.65%	0.00%	python3	python3.12	[.] Py_Exit
+	1.65%	0.00%	python3	python3.12	[.] Py_FinalizeEx
+	1.50%	0.00%	python3	python3.12	[.] PyObject_VectorcallDict
+	1.38%	0.00%	python3	python3.12	[.] 0x00000000000605964

Ignoring parent start frames, one function that took up quite a lot of time was *PyEvalEvalFrameDefault* (took up 76.27% of the total time itself).

Q6: Flamegraph



From the flamegraph, we can see that `PyEval_EvalFrameDefault` is the function that took up a lot of time on its own. Its children did not take much time yet the total time spent executing this function was significant (evident from width), showing much time was spent executing the function itself.

This shows that much time is spent executing python code. The likely culprit is the $O(n^3)$ loop used in `matmul_slow` to do the evaluation of matrix product.

Q7: Improvements to `matmul_slow`

Looking at `matmul_fast`, there's 3 functions that implement a faster form of matrix multiplications:

1. Matmul_fast1
2. Matmul_fast2
3. Matmul_fast3

They all make use of the idea of locality to reduce cache misses. In these implementations, we fetch entire rows from each matrix at once and store it in a new temp variable. These temp variables are also accessed very frequently in the inner loop. This ensures that they are stored in the caches and thus reduces cache miss, improving speed.